

Mastering the Continuous Fourier Series (CFS)

Comprehensive Property Testing & Implementation Guide

Introduction & Setup

To verify Fourier Series properties programmatically, we must extract the coefficients from your `FourierEpicycles` class (which stores them in a dictionary) into a sorted NumPy array. We also need a standard Mean Squared Error (MSE) function.

Add this setup block to the top of your test script tomorrow:

```
import numpy as np
import scipy.integrate

def calculate_mse(arr1, arr2):
    """Computes the Mean Squared Error between two complex arrays."""
    return np.mean(np.abs(arr1 - arr2)**2)

def get_coeff_array(fs_obj):
    """Extracts the dictionary of coefficients into a sorted 1D complex array."""
    sorted_keys = sorted(fs_obj.coefs.keys())
    return np.array([fs_obj.coefs[k] for k in sorted_keys], dtype=complex)

# Assume `t` and `z` are loaded from svg_utils, and `N_HARMONICS` is defined.
# fs_orig = FourierEpicycles(t, z, N_HARMONICS)
# fs_orig.calculate_all_coefficients()
# c_orig = get_coeff_array(fs_orig)
```

1 Linearity

Problem Statement: Given a signal $z(t)$, construct a new signal $y(t) = A \cdot z(t) + B$, where A and B are complex constants. Verify the linearity property.

Theory: If $x(t) \leftrightarrow a_k$ and $y(t) \leftrightarrow b_k$, then $Ax(t) + By(t) \leftrightarrow Aa_k + Bb_k$. Adding a constant B is equivalent to adding a DC signal. Therefore, only the 0-th harmonic (c_0) is affected by B . All other harmonics are simply scaled by A .

```
A = 2.5 + 1j
B = 3.0 - 2.0j
y_signal = A * z + B

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

# Theoretical Prediction
c_theo = A * c_orig
zero_idx = N_HARMONICS # In an array from -N to N, index N is the 0th harmonic
c_theo[zero_idx] += B

print(f"Linearity MSE: {calculate_mse(c_y, c_theo)}")
```

Explanation: We scale the original coefficient array by A . Because B is a constant (frequency = 0), we add it *only* to the index corresponding to $n = 0$.

2 Time Shifting

Problem Statement: Shift the signal $z(t)$ in time by t_0 . Verify the time-shifting property.

Theory: $x(t - t_0) \leftrightarrow c_n e^{-jn\omega_0 t_0}$. Delaying a signal does not change its shape, so the magnitude of the coefficients $|c_n|$ remains unchanged. However, to reconstruct the delayed shape, each frequency component must be phase-shifted by an amount proportional to its frequency ($-n\omega_0 t_0$).

```
t0 = fs_orig.T / 4.0
dt = t[1] - t[0]
shift_idx = int(t0 / dt)

# Shift the array. Since it's a closed interval (t[0] == t[-1]),
# we roll the array excluding the last element, then append the first element to
# the end.
z_open = z[:-1]
y_open = np.roll(z_open, shift_idx)
y_signal = np.append(y_open, y_open[0])

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

# Theoretical Prediction
n_arr = np.arange(-N_HARMONICS, N_HARMONICS + 1)
c_theo = c_orig * np.exp(-1j * n_arr * fs_orig.omega * t0)

print(f"Time Shift MSE: {calculate_mse(c_y, c_theo)}")
```

Explanation: We use `np.roll` to shift the discrete array. Because your assignment uses a *closed* interval where $z[-1] == z[0]$, we must temporarily remove the duplicate endpoint, roll the array, and then restore the endpoint to maintain perfect periodicity for `np.trapezoid`.

3 Frequency Shifting (Modulation)

Problem Statement: Multiply the signal by a complex exponential $y(t) = e^{jM\omega_0 t} z(t)$. Verify the frequency-shifting property.

Theory: $e^{jM\omega_0 t} x(t) \leftrightarrow c_{n-M}$. Multiplying by a complex exponential in the time domain shifts the entire frequency spectrum to the right by M harmonics.

```
M = 3
y_signal = z * np.exp(1j * M * fs_orig.omega * t)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

# Theoretical Prediction
# The computed array c_y is shifted right by M.
# We compare the overlapping valid regions.
c_y_valid = c_y[M:]          # Computed shifted coefficients
c_theo_valid = c_orig[:-M]    # Original coefficients

print(f"Frequency Shift MSE: {calculate_mse(c_y_valid, c_theo_valid)}")
```

Explanation: Because we only compute a finite number of harmonics ($-N$ to N), shifting the spectrum drops the last M elements and introduces M unknown elements at the beginning. We slice the arrays to compare only the overlapping, known regions.

4 Time Reversal & Conjugation

Problem Statement: Create a time-reversed signal $y(t) = z(-t)$ and a conjugated signal $w(t) = z^*(t)$. Verify their properties.

Theory: Time Reversal: $x(-t) \leftrightarrow c_{-n}$. Conjugation: $x^*(t) \leftrightarrow c_{-n}^*$. Reversing time reverses the direction of the rotating epicycles, mapping harmonic n to $-n$. Conjugating the signal flips the shape over the x-axis, which conjugates the coefficients and reverses their order.

```
# Time Reversal
y_signal = np.zeros_like(z)
y_signal[0] = z[0]
y_signal[1:] = z[:0:-1] # Reverse everything except the t=0 point

# Conjugation
w_signal = np.conj(z)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

fs_w = FourierEpicycles(t, w_signal, N_HARMONICS)
fs_w.calculate_all_coefficients()
c_w = get_coeff_array(fs_w)

# Theoretical Predictions
c_theo_y = c_orig[::-1] # Reverse the array (n becomes -n)
c_theo_w = np.conj(c_orig[::-1]) # Conjugate and reverse

print(f"Time Reversal MSE: {calculate_mse(c_y, c_theo_y)}")
print(f"Conjugation MSE: {calculate_mse(c_w, c_theo_w)}")
```

Explanation: To reverse the signal array properly on a closed interval $[0, T]$, the point at $t = 0$ stays at $t = 0$. The point at $t = \Delta t$ swaps with $t = T - \Delta t$. We achieve this by keeping index 0 fixed and reversing the slice from index 1 to the end.

5 Time Scaling

Problem Statement: Compress the signal in time by a factor of $\alpha = 2$, so $y(t) = z(2t)$. Verify the time-scaling property.

Theory: $x(\alpha t) \leftrightarrow c_n$. If you play a signal twice as fast, the shape of the signal does not change, so the Fourier coefficients c_n remain exactly the same! However, the fundamental frequency changes from ω_0 to $\alpha\omega_0$, and the period changes from T to T/α .

```
alpha = 2.0
T_new = fs_orig.T / alpha
# Create a new time array that spans the new, shorter period
t_new = np.linspace(0, T_new, len(t))

# The signal array z remains exactly the same!
fs_y = FourierEpicycles(t_new, z, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)
```

```
print(f"Time Scaling MSE: {calculate_mse(c_y, c_orig)}")
print(f"Original Omega: {fs_orig.omega:.2f}, New Omega: {fs_y.omega:.2f}")
```

Explanation: This is an OOP trick. We do not modify the signal array $\mathbf{z}$. We simply pass a new time array $\mathbf{t_new}$ that spans half the time. The `FourierEpicycles` class automatically recalculates the new T and ω . The coefficients remain identical.

6 Differentiation

Problem Statement: Compute the derivative of the signal $y(t) = \frac{d}{dt}z(t)$. Verify the differentiation property.

Theory: $\frac{dx(t)}{dt} \leftrightarrow jn\omega_0 c_n$. Taking the derivative amplifies high frequencies (because they change faster, hence multiplication by $n\omega_0$) and applies a 90° phase shift (represented by j).

```
dt = t[1] - t[0]
y_signal = np.gradient(z, dt)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

# Theoretical Prediction
n_arr = np.arange(-N_HARMONICS, N_HARMONICS + 1)
c_theo = 1j * n_arr * fs_orig.omega * c_orig

print(f"Differentiation MSE: {calculate_mse(c_y, c_theo)}")
```

Explanation: We use `np.gradient` to numerically differentiate the signal. It is superior to `np.diff` because it preserves the array length and handles boundary conditions automatically.

7 Integration

Problem Statement: Compute the integral of the signal $y(t) = \int_{-\infty}^t x(\tau)d\tau$. Verify the integration property.

Theory: $\int x(t)dt \leftrightarrow \frac{c_n}{jn\omega_0}$. Integration attenuates high frequencies (division by $n\omega_0$). **Critical Condition:** This property is only valid if the signal has zero mean ($c_0 = 0$). If $c_0 \neq 0$, integrating a constant produces a ramp function, which is not periodic!

```
# 1. Remove the DC component to ensure periodicity after integration
c0 = fs_orig.coefs[0]
z_zero_mean = z - c0

# 2. Integrate numerically
y_signal = scipy.integrate.cumulative_trapezoid(z_zero_mean, t, initial=0)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

# Theoretical Prediction
n_arr = np.arange(-N_HARMONICS, N_HARMONICS + 1)
c_theo = np.zeros_like(c_orig, dtype=complex)

for i, n in enumerate(n_arr):
    if n != 0:
        c_theo[i] = c_orig[i] / (1j * n * fs_orig.omega)
    else:
```

```

        # The DC component of the integral depends on the constant of integration.
        # We ignore it for the MSE calculation by setting it equal to the computed
        one.
        c_theo[i] = c_y[i]

print(f"Integration MSE: {calculate_mse(c_y, c_theo)}")

```

Explanation: We subtract c_0 from the signal before integrating. We use `cumulative_trapezoid` with `initial=0` to maintain the array shape. In the theoretical calculation, we must explicitly skip $n = 0$ to avoid a `ZeroDivisionError`.

8 Conjugate Symmetry & Even/Odd Decomposition

Problem Statement: Extract the real part of the signal $x(t) = \Re\{z(t)\}$. Verify conjugate symmetry. Then, decompose $x(t)$ into Even and Odd parts and verify their coefficient properties.

Theory: If $x(t)$ is real, $c_n = c_{-n}^*$ (Conjugate Symmetry). If $x(t)$ is real and Even, c_n is purely real. If $x(t)$ is real and Odd, c_n is purely imaginary.

```

x_real = z.real
fs_real = FourierEpicycles(t, x_real, N_HARMONICS)
fs_real.calculate_all_coefficients()
c_real = get_coeff_array(fs_real)

# 1. Verify Conjugate Symmetry
# c_real[::-1] reverses the array, mapping n to -n
mse_sym = calculate_mse(c_real, np.conj(c_real[::-1]))
print(f"Conjugate Symmetry MSE: {mse_sym}")

# 2. Even/Odd Decomposition
x_rev = np.zeros_like(x_real)
x_rev[0] = x_real[0]
x_rev[1:] = x_real[:0:-1]

x_even = (x_real + x_rev) / 2.0
x_odd = (x_real - x_rev) / 2.0

fs_even = FourierEpicycles(t, x_even, N_HARMONICS)
fs_even.calculate_all_coefficients()
c_even = get_coeff_array(fs_even)

fs_odd = FourierEpicycles(t, x_odd, N_HARMONICS)
fs_odd.calculate_all_coefficients()
c_odd = get_coeff_array(fs_odd)

# Theoretical: Even part coeffs = Real part of c_real. Odd part coeffs = j * Imag
# part.
print(f"Even Part MSE: {calculate_mse(c_even, c_real.real)}")
print(f"Odd Part MSE: {calculate_mse(c_odd, 1j * c_real.imag)}")

```

Explanation: We isolate the real part of the SVG drawing. We verify symmetry by comparing the array to its reversed, conjugated self. We then manually construct the even and odd signals in the time domain and verify that their coefficients match the real and imaginary parts of the original coefficients.

9 Periodic Convolution

Problem Statement: Compute the periodic convolution of $z(t)$ with a test signal $y(t) = \sin(\omega_0 t)$. Verify the convolution property.

Theory: $x(t) \circledast y(t) \leftrightarrow T \cdot c_n d_n$. Convolution in the time domain equals multiplication in the frequency domain, scaled by the period T .

```
y_sig = np.sin(fs_orig.omega * t)

# Manual O(N^2) Periodic Convolution
w_sig = np.zeros_like(t, dtype=complex)
y_rev = np.zeros_like(y_sig)
y_rev[0] = y_sig[0]
y_rev[1:] = y_sig[:0:-1] # y(-tau)

for i in range(len(t)):
    y_shifted = np.roll(y_rev, i) # y(t - tau)
    w_sig[i] = np.trapezoid(z * y_shifted, t)

fs_y = FourierEpicycles(t, y_sig, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

fs_w = FourierEpicycles(t, w_sig, N_HARMONICS)
fs_w.calculate_all_coefficients()
c_w = get_coeff_array(fs_w)

# Theoretical Prediction
c_theo = fs_orig.T * c_orig * c_y

print(f"Periodic Convolution MSE: {calculate_mse(c_w, c_theo)}")
```

Explanation: Because built-in convolution functions use FFTs (which are forbidden), we implement the convolution integral manually. We reverse $y(t)$ to get $y(-\tau)$, shift it by i steps to get $y(t - \tau)$, multiply by $z(\tau)$, and integrate using `np.trapezoid`.

10 Multiplication

Problem Statement: Multiply two signals in the time domain $w(t) = z(t) \cdot \cos(2\omega_0 t)$. Verify the multiplication property.

Theory: $x(t)y(t) \leftrightarrow \sum_{l=-\infty}^{\infty} c_l d_{n-l}$. Multiplication in the time domain equals discrete convolution in the frequency domain.

```
y_sig = np.cos(2 * fs_orig.omega * t)
w_sig = z * y_sig

fs_y = FourierEpicycles(t, y_sig, N_HARMONICS)
fs_y.calculate_all_coefficients()
c_y = get_coeff_array(fs_y)

fs_w = FourierEpicycles(t, w_sig, N_HARMONICS)
fs_w.calculate_all_coefficients()
c_w = get_coeff_array(fs_w)

# Theoretical Prediction: Discrete convolution of the coefficient arrays
# mode='same' keeps the output array the same size as the inputs (-N to N)
c_theo = np.convolve(c_orig, c_y, mode='same')

print(f"Multiplication MSE: {calculate_mse(c_w, c_theo)}")
```

Explanation: We multiply the signals element-wise in time. In the frequency domain, we use `np.convolve` to convolve the two coefficient arrays.

11 Parseval's Relation

Problem Statement: Verify Parseval's theorem for the signal $z(t)$.

Theory: $\frac{1}{T} \int_0^T |x(t)|^2 dt = \sum_{n=-\infty}^{\infty} |c_n|^2$. The total power of the signal in the time domain must equal the sum of the power of all its frequency components.

```
# Time domain power
power_time = np.trapezoid(np.abs(z)**2, t) / fs_orig.T

# Frequency domain power
power_freq = np.sum(np.abs(c_orig)**2)

print(f"Parseval's - Time Power: {power_time:.4f}")
print(f"Parseval's - Freq Power: {power_freq:.4f}")
print(f"Difference (Energy in truncated harmonics): {power_time - power_freq:.6f}")
```

Explanation: We integrate the squared magnitude of the signal over time and divide by T . We compare this to the sum of the squared magnitudes of the coefficients. The difference will not be exactly zero because we are only summing up to N harmonics, leaving a tiny amount of high-frequency energy unaccounted for.